Lab7 – Formal

The final lab session in the PV course focuses on functional verification. The objective of this session is to conduct a formal verification of a basic FSM module, which represents a typical application of formal verification methodologies. Recall that the purpose of Formal Property Verification (FPV) is to develop a set of assert and assume statements that accurately define the module's specification and appropriately constrain the environment.

The initial plan for this session was to utilize the Siemens tools included in the Questa Formal package. However, these tools are not yet available on the FIB servers due to various factors. Therefore, we will proceed with a simple, free, open-source alternative for this session, acknowledging that these options may present certain limitations.

An initial example (counter.sv) is provided, containing both the RTL code and the associated assert/assume statements. While these elements are typically organized into separate modules or even distinct files, for the sake of simplicity and due to tool constraints, they have been consolidated into a single module in this instance. This example is intended to help you familiarize yourself with the process. Once you are comfortable, you will be responsible for incorporating formal asserts and assumes into the packet.sv module.

1. Get Ready for this Lab

In the web of the course at https://docencia.ac.upc.edu/master/MIRI/PV/labs.html, for session Lab7, you will find a package called packet.tgz, which contains the example for the counter (example/counter.sv) as well as this session's design (packet.sv).

Begin with the provided example. Open the file to locate the RTL of the design, which features a simple counter capped at MAX_AMOUNT, as well as the list of assume/assert statements intended for use with the formal verification tool.

Tool: EBMC

EBMC is a free, open-source formal verification (FV) tool for hardware designs. It can read Verilog 2005, SystemVerilog 2017. It includes both bounded and (despite its name) unbounded Model Checking engines, i.e., it can both discover bugs and prove the absence of bugs.

More information can be found on their webpage: EBMC: The Enhanced Bounded Model Checker

There are two ways to install the tool on your machine: you can either use the Linux package or build it from source. For this lab session, I suggest going with the first option.

Option #1: Install EBMC from package (assuming no root privileges)

For Ubuntu/Debian:

```
$ mkdir -p software/ebmc && cd software/ebmc
$ wget https://github.com/diffblue/hw-cbmc/releases/download/ebmc-5.8/ebmc_5.8_amd64.deb
$ dpkg-deb -x ebmc_5.8_amd64.deb .
$ export PATH=$PWD/usr/bin:$PATH
```

For Fedora/CentOS/SUSE (use this in FIB's machines)

```
$ mkdir -p software/ebmc && cd software/ebmc
$ wget https://github.com/diffblue/hw-cbmc/releases/download/ebmc-5.8/ebmc-5.8-1.x86_64.rpm
$ rpm2cpio ebmc-5.8-1.x86_64.rpm | cpio -idmv
$ setenv PATH $PWD/usr/bin:$PATH
```

Option #2: Install EBMC from source

```
$ apt-get install g++ gcc flex bison make git curl patch
$ git clone https://github.com/diffblue/hw-cbmc.git
$ cd hw-cbmc
$ git checkout ebmc-5.8
$ git submodule init
$ git submodule update ← Slow …
$ make -C lib/cbmc/src minisat2-download
$ make -C src -j 2
$ export PATH=$PWD/src/ebmc;$PATH
```

Run the example

```
$ ebmc –version
5.8 (ebmc-5.8)
$ ebmc counter.sv -D FORMAL --top counter --bound 20 --systemverilog --trace --waveform --vcd counter.vcd
```

Review and assess the log output. Then, intentionally introduce errors into the design to observe how the tool responds. For instance:

- `count <= MAX_AMOUNT)`
- `count <= count + 1'b2`

In these cases—assuming the assertions are accurate and complete—in addition to logging the counterexample test vector, the tool also generates a waveform in counter.vcd, which you can open with gtkwave.

Please observe the specific details presented in the FORMAL section of this example. Formal constraint-based tools are typically very picky and require strict adherence to the initial state parameters and reset signals:

- Note that $past function is not defined for the first cycle (there is no such *past*).
  The past_valid signal is used to avoid this case.
- Note also that main properties ignore the reset cycles, both current and previous.

2. Design: Packet

Once you have reviewed the given example, begin your initial formal verification. Below is the module specification, which you can use alongside the code from the packet.sv file. Please make sure to place your FORMAL code in the specified section.

Overview

The packet module implements a finite state machine (FSM) that models the reception of a packet through successive stages: header, payload, checksum, and completion. It generates status flags (`valid_pkt`, `error_pkt`) to indicate whether a packet was successfully received or failed due to checksum errors.

Interface

Signal	Direction	Description
`clk`	Input	System clock. All state transitions occur on the rising edge.
`reset`	Input	Synchronous reset. Forces the FSM back to IDLE and clears outputs.
`start_pkt`	Input	Initiates packet reception. Valid only in IDLE.
`hdr_done`	Input	Signals completion of header processing. Valid only in HEADER.
`payload_done`	Input	Signals completion of payload processing. Valid only in PAYLOAD.
`chk_ok`	Input	Indicates checksum verification passed. Valid only in CHECKSUM.
`ckh_fail`	Input	Indicates checksum verification failed. Valid only in CHECKSUM.
`abort`	Input	Aborts packet reception at any stage, returning FSM to IDLE.
`valid_pkt`	Output	Pulses high when a packet is successfully received (transition from CHECKSUM to DONE).
`error_pkt`	Output	Pulses high when checksum fails (transition from CHECKSUM to IDLE).
`state[2:0]`	Output	Encoded FSM state.

State Encoding

- IDLE (0): Waiting for packet start.
- HEADER (1): Processing header.
- PAYLOAD (2): Processing payload.
- CHECKSUM (3): Verifying checksum.
- DONE (4): Packet reception complete; automatically returns to IDLE.

Behavior

1. On reset, FSM enters IDLE and clears outputs.
2. In IDLE, assertion of **`start_pkt`** moves FSM to HEADER.
3. In HEADER, assertion of **`hdr_done`** moves FSM to PAYLOAD.
4. In PAYLOAD, assertion of **`payload_done`** moves FSM to CHECKSUM.
5. In CHECKSUM:
    a. If **`chk_ok`** is asserted, FSM moves to DONE and raises **`valid_pkt`**.
    b. If **`chk_fail`** is asserted, FSM returns to IDLE and raises **`error_pkt`**.
6. In DONE, FSM automatically returns to IDLE on the next cycle.
7. At any stage, assertion of **`abort`** forces FSM back to IDLE with outputs cleared.

3. What to Turn In

Please provide:

(i) A single PDF document with the following:

1. Please indicate how many hours you spent on this lab. This will not affect your grade, but will be helpful for calibrating the workload for the future.
2. (1 pt) It's always helpful to begin with the FSM graph. Since the architect did not include it in the specification due to oversight, you should create the graph yourself using the provided specification text.
3. (7 pts) Write the assume/assert statements for packet.sv (where it says "//FORMAL STATEMENTS BELOW")
    a. List (with brief description) the assume/assert statements you plan to add.
    b. Add them with SVA.

    Execute the tool. The design is expected to be correct, so run the tool and dump the output in the document.
4. (2 pts) Intentionally introduce a bug into your design, execute the program, and include its output in your document. Then, explain the results using your own words (e.g., how many counter-examples are provided, how is the waveform, etc.)